

Atividade 3: Comunicação em Sistemas Distribuídos

Um Estudo Comparativo de Chat Distribuído com Sockets TCP, RPC (gRPC) e Java RMI

Matheus Antunes Freire

Pedro Bazzarella Pereira

Disciplina de Sistemas Distribuídos

Outubro de 2026

Resumo

Este trabalho apresenta o projeto, a implementação e a análise comparativa de uma aplicação distribuída de chat cliente-servidor desenvolvida em Java (JDK 26) sob três paradigmas distintos de comunicação em rede: **Sockets TCP**, **Remote Procedure Call (RPC via gRPC)** e **Remote Method Invocation (Java RMI)**. Para garantir máxima coesão e isolamento tecnológico, a aplicação foi estruturada seguindo os princípios da Arquitetura Hexagonal (*Ports & Adapters*) em conjunto com o padrão de projeto *Strategy*, permitindo a substituição transparente do mecanismo de transporte em tempo de execução via argumentos de linha de comando. Adicionalmente, toda a infraestrutura foi containerizada através do Docker e Docker Compose, assegurando reprodutibilidade e portabilidade. O relatório detalha as características arquiteturais de cada versão, avalia o nível de abstração, desempenho e complexidade, e discute as dificuldades práticas superadas durante o desenvolvimento.

Sumário

1	Introdução e Definição da Aplicação	3
1.1	Justificativa da Aplicação Escolhida	3
2	Arquitetura Geral do Sistema	3
2.1	Estrutura de Camadas	3
2.2	Containerização com Docker	4
3	Arquitetura e Implementação das Três Versões	5
3.1	Versão 1: Sockets TCP	5
3.1.1	Mecanismo de Comunicação	5
3.2	Versão 2: RPC com gRPC e Protocol Buffers	5
3.2.1	Definição do Contrato (<code>chat.proto</code>)	5
3.2.2	Mecanismo de Comunicação	6
3.3	Versão 3: Java RMI (Remote Method Invocation)	6
3.3.1	Mecanismo de Comunicação	6
4	Comparação Técnica Entre as Três Abordagens	7
4.1	Nível de Abstração Oferecido	7
4.2	Facilidade de Uso e Complexidade	7
4.3	Casos de Uso Práticos	9
5	Dificuldades Encontradas e Soluções Adotadas	9
6	Conclusão	10

1 Introdução e Definição da Aplicação

O desenvolvimento de sistemas distribuídos contemporâneos impõe desafios contínuos relativos ao desacoplamento entre módulos de software e à escolha do modelo de comunicação adequado para os requisitos de latência, escalabilidade, interoperabilidade e manutenibilidade.

Com o intuito de explorar pragmaticamente tais aspectos, este projeto propôs a implementação de um **Sistema de Chat Distribuído Cliente-Servidor Multi-Dispositivo**, no qual múltiplos clientes podem se conectar simultaneamente a um servidor central, enviar mensagens de texto e recebê-las de forma assíncrona e contínua via *broadcast*.

1.1 Justificativa da Aplicação Escolhida

A aplicação de chat foi selecionada por demandar simultaneamente características fundamentais de sistemas distribuídos:

1. **Bidirecionalidade Contínua:** O cliente não apenas requisita dados, mas deve ser capaz de receber mensagens assíncronas do servidor no momento exato em que são publicadas por outros clientes.
2. **Concorrência e Compartilhamento de Estado:** O servidor deve gerenciar múltiplos canais ativos concorrentemente na memória RAM sem corromper o estado dos clientes conectados.
3. **Heterogeneidade Conceitual:** O problema de transmitir uma mensagem para múltiplos participantes evidencia claramente como cada tecnologia lida com fluxo de bytes (Sockets), chamadas remotas de procedimentos (RPC) e referências a objetos remotos (RMI).

2 Arquitetura Geral do Sistema

Para viabilizar que as três tecnologias coexistissem no mesmo projeto sem duplicação de lógica de apresentação e domínio, adotou-se a **Arquitetura Hexagonal (*Ports & Adapters*)** combinada com o padrão de projeto ***Strategy***.

2.1 Estrutura de Camadas

- **Core de Domínio (*Domain*):** Contém as entidades nucleares do sistema, livres de qualquer dependência externa ou tecnológica:
 - **Message:** Representa a mensagem de texto contendo remetente (*sender*), conteúdo textual (*content*), carimbo de data/hora (*timestamp*) e destinatário

- (*recipient*). Implementa `Serializable` para atender à serialização nativa do Java.
 - **User**: Encapsula o identificador ou nome de usuário (*username*) do participante conectado.
- **Portas (*Ports*)**: Definem os contratos abstratos de comunicação:
 - **ChatServerPort**: Contrato do servidor, exigindo operações de ciclo de vida (`start`, `stop`) e disseminação (`broadcast`).
 - **ChatClientPort**: Contrato do cliente, exigindo operações de conexão (`connect`), envio (`sendMessage`) e encerramento (`disconnect`).
 - **MessageListener**: Interface funcional atuando como *Observer*, pela qual as camadas de rede notificam a interface de usuário sempre que uma nova mensagem for entregue.
- **Adaptadores de Rede (*Adapters*)**: Implementações concretas das portas para cada protocolo: `sockets`, `grpc` e `rmi`.
- **Fábrica de Rede (*NetworkFactory*)**: Ponto centralizado de injeção que recebe a *string* correspondente ao protocolo via CLI (`"socket"`, `"grpc"` ou `"rmi"`) e instancia o adaptador correto.
- **Interface de Linha de Comando (*CLI UI*)**: Implementa `MessageListener` para exibir mensagens na saída padrão sem bloquear o laço de leitura de entrada do usuário via terminal.

2.2 Containerização com Docker

Conforme decisão técnica do projeto, toda a aplicação foi containerizada empregando o **JDK 26** em um *build multi-stage*:

- No estágio de compilação (*builder*), o Maven gera os stubs a partir da definição do Protocol Buffers (`.proto`) e empacota todas as dependências em um *shaded JAR* executável.
- No estágio final de execução (*runtime*), uma imagem enxuta baseada em `eclipse-temurin:26` expõe as portas de serviço e fornece os *entrypoints* unificados para o servidor (`ServerApp`) e clientes (`ClientApp`).
- O `docker-compose.yml` orquestra instâncias independentes dos três servidores e facilita a execução dos clientes no modo interativo.

3 Arquitetura e Implementação das Três Versões

3.1 Versão 1: Sockets TCP

A versão baseada em sockets emprega a API nativa `java.net.ServerSocket` e `java.net.Socket`, operando na camada de transporte sobre o protocolo TCP (*Transmission Control Protocol*), garantindo entrega confiável, ordenada e com verificação de erros.

3.1.1 Mecanismo de Comunicação

- **Servidor (SocketServerAdapter):** Escuta requisições na porta designada (padrão 8080). Ao aceitar uma conexão, delega o canal para uma thread dedicada executada por um `CachedThreadPool` gerenciando instâncias de `SocketClientHandler`.
- **Serialização:** O transporte de objetos `Message` entre cliente e servidor foi realizado via `ObjectOutputStream` e `ObjectInputStream`. Após o envio de cada objeto, executa-se `flush()` e `reset()` para descarregar o buffer do socket e limpar o cache de instâncias do fluxo de serialização.
- **Recepção Assíncrona no Cliente (SocketClientAdapter):** Ao conectar, o cliente inicia uma thread em segundo plano (*daemon*) dedicada exclusivamente a ler o `ObjectInputStream` e acionar o callback `MessageListener.onMessageReceived()`.

3.2 Versão 2: RPC com gRPC e Protocol Buffers

O paradigma RPC (*Remote Procedure Call*) abstrai a rede de modo que invocar um método em um servidor remoto seja sintaticamente idêntico a chamar uma função local. Para esta versão, utilizou-se o framework de alta performance **gRPC** sobre o protocolo **HTTP/2**, com especificação formal via **Protocol Buffers v3**.

3.2.1 Definição do Contrato (chat.proto)

O contrato de comunicação foi definido formalmente:

```
syntax = "proto3";
package com.pedrobazza.distsyschat.adapters.grpc;

service ChatService {
    rpc StreamMessages(ConnectRequest) returns (stream
        ChatMessage);
    rpc SendMessage(ChatMessage) returns (SendResponse);
    rpc Disconnect(DisconnectRequest) returns (DisconnectResponse
    );
}
```

```

message ConnectRequest { string username = 1; }
message ChatMessage {
    string sender = 1;
    string content = 2;
    int64 timestamp = 3;
    string recipient = 4;
}
message SendResponse { bool success = 1; string message = 2; }
message DisconnectRequest { string username = 1; }
message DisconnectResponse { bool success = 1; }

```

3.2.2 Mecanismo de Comunicação

- **Streaming Unidirecional Servidor-Cliente (*Server-Streaming RPC*):** O método remoto `StreamMessages` permite que o cliente envie seu nome de usuário e receba um fluxo contínuo de eventos (`stream ChatMessage`) via `StreamObserver`. O servidor mantém as instâncias ativas de `StreamObserver` em memória para enviar broadcasts instantâneos.
- **Invocação Unária (*Unary RPC*):** Os métodos `SendMessage` e `Disconnect` realizam chamadas diretas com retorno estruturado, manipuladas pelo stub bloqueante (`ChatServiceBlockingStub`).

3.3 Versão 3: Java RMI (Remote Method Invocation)

O Java RMI é a tecnologia proprietária da plataforma Java para comunicação baseada em **Objetos Distribuídos**. Diferente de Sockets e RPC convencionais, o RMI permite invocar métodos em instâncias de objetos que residem em uma JVM remota, com transmissão transparente de parâmetros e exceções por meio de stubs e skeletons gerados dinamicamente.

3.3.1 Mecanismo de Comunicação

- **Interface Remota (`RmiRemoteService`):** Estende `java.rmi.Remote` e declara os métodos remotos que o servidor disponibiliza:

```

public interface RmiRemoteService extends Remote {
    interface ClientCallback extends Remote {
        void receiveMessage(Message message) throws
            RemoteException;
    }
    void registerClient(String username, ClientCallback
        callback) throws RemoteException;
}

```

```
void unregisterClient(String username) throws
    RemoteException;
void postMessage(Message message) throws RemoteException;
}
```

- **Registro de Nomes (*RMI Registry*):** O servidor instancia um serviço remoto (`RmiRemoteServiceImpl`) estendendo `UnicastRemoteObject` e registra sua referência no *RMI Registry* na porta 1099 através do nome chave "ChatService".
- **Callback Remoto para Broadcast:** Como o RMI é classicamente orientado a requisições do cliente para o servidor, a bidirecionalidade do chat exigiu a criação de um **Callback Remoto**: o cliente exporta um objeto remoto anônimo (`ClientCallback`) e envia essa referência remota ao servidor no momento do registro. Ao receber uma mensagem, o servidor itera sobre a coleção de stubs de callback dos clientes e invoca remotamente o método `receiveMessage(message)` de cada um.

4 Comparação Técnica Entre as Três Abordagens

A Tabela 1 resume as principais características comparativas entre Sockets TCP, gRPC e Java RMI no contexto do desenvolvimento da aplicação de chat.

4.1 Nível de Abstração Oferecido

- **Sockets:** Exige que o desenvolvedor gerencie explicitamente todos os aspectos da comunicação: abertura e fechamento de sockets, instanciação de buffers de entrada/saída, tratamento de conexões prematuramente encerradas e criação de laços com multithreading.
- **gRPC:** Fornece abstração focada no negócio. O desenvolvedor interage com classes fortemente tipadas geradas pelo compilador, não precisando lidar com sockets TCP ou detalhes de transporte HTTP/2.
- **RMI:** Oferece a mais alta transparência sintática para desenvolvedores Java, pois a invocação remota parece uma chamada de método Java comum (`service.postMessage(msg)`), encapsulando a complexidade da rede nos stubs.

4.2 Facilidade de Uso e Complexidade

Embora Sockets não exijam bibliotecas externas, a complexidade de manter estados concorrentes e gerenciar threads de leitura/escrita eleva o risco de vazamentos de recursos ou travamentos por concorrência.

Tabela 1: Comparação entre Sockets, gRPC e Java RMI.

Critério	Sockets TCP	gRPC (RPC)	Java RMI
Nível de Abstração	Baixo (fluxo de bytes bruto ou objetos serializados).	Alto (procedimentos remotos com contrato formal IDL).	Muito Alto (invocação de métodos em instâncias distribuídas).
Protocolo de Transporte	TCP puro com portas customizadas.	HTTP/2 com multiplexação de conexões.	TCP / JRMP (<i>Java Remote Method Protocol</i>).
Formato dos Dados	Binário via serialização Java nativa.	Binário compacto via Protocol Buffers.	Binário via serialização nativa de classes Java.
Bidirecionalidade	Manual: controle de threads separadas de leitura e escrita.	Nativa: suporte embutido a streaming unidirecional e bidirecional.	Indireta: requer exportação de objetos de <i>callback</i> pelo cliente.
Interoperabilidade	Média (depende da especificação manual do protocolo).	Excelente (agnóstico a plataformas e linguagens).	Nula (restrito e acoplado ao ecossistema JVM/Java).
Complexidade de Código	Alta para sincronização e tratamento de exceções de rede.	Média (requer etapa de compilação de stubs, mas simplifica o código).	Baixa para chamadas simples; Média/Alta para gerenciar callbacks e rede.

O gRPC possui uma curva de aprendizado inicial vinculada à sintaxe do Protobuf e à configuração dos plugins Maven de geração de código. No entanto, uma vez configurado, o desenvolvimento torna-se extremamente previsível e robusto.

O RMI apresenta a menor complexidade de código inicial para comunicação básica em Java, mas expõe dificuldades relevantes em ambientes modernos com contêineres e NAT, uma vez que requer configuração precisa de hostnames públicos e portas dinâmicas para os stubs exportados.

4.3 Casos de Uso Práticos

- **Sockets TCP:** Indicado para jogos online em tempo real, sistemas embarcados com hardware limitado, protocolos proprietários de telecomunicações e serviços em que o controle sobre cada byte transmitido é imperativo.
- **gRPC:** Padrão da indústria para arquiteturas de microserviços em nuvem, comunicação entre serviços de backend distribuídos em diferentes linguagens (ex: Java, Go, Python, C++) e aplicações móveis que demandam streaming eficiente e baixo consumo de banda.
- **Java RMI:** Adequado para sistemas legados corporativos puramente Java (JEE/EJB) operando dentro de redes locais corporativas fechadas, onde a homogeneidade de tecnologia é garantida.

5 Dificuldades Encontradas e Soluções Adotadas

Durante o desenvolvimento da Etapa 1, alguns desafios técnicos foram identificados e solucionados:

1. **Conflito de Provedores SPI no Empacotamento gRPC com Maven Shade:** Ao gerar o JAR executável contendo todas as dependências (*fat JAR*), o gRPC falhava na resolução de nomes do canal com a mensagem de que nenhum `NameResolverProvider` havia sido encontrado. *Solução:* Adicionou-se o `ServicesResourceTransformer` à configuração do `maven-shade-plugin` no `pom.xml`, garantindo que todos os arquivos declaratórios em `META-INF/services/` fossem mesclados ao invés de sobrescritos.
2. **Roteamento de Callbacks RMI em Rede Docker Bridge:** Em contêineres Docker, o RMI Registry registrava o endereço interno efêmero do contêiner. Quando o cliente tentava registrar o callback ou o servidor tentava invocar métodos de volta no cliente, ocorria travamento por tempo limite de conexão (*timeout*). *Solução:* Configurou-se a propriedade de sistema `java.rmi.server.hostname` do Java RMI com suporte à leitura de variáveis de ambiente (`RMI_HOST`), permitindo que os

contêineres se comuniquem corretamente através do DNS interno da rede do Docker Compose.

3. **Finalização Graciosa de Processos Multithread:** O cliente RMI mantinha threads ativas mesmo após o encerramento do chat, impedindo a finalização do processo do terminal. *Solução:* Implementou-se o desregistro explícito com `unexportObject(callback, true)` associado à chamada deliberada de encerramento de threads residuais no fechamento da interface de linha de comando.

6 Conclusão

A execução prática deste projeto atingiu plenamente os objetivos propostos na Atividade 3. A implementação das três versões de chat com Sockets TCP, gRPC e Java RMI sobre a mesma aplicação evidenciou claramente a evolução dos modelos de comunicação distribuída: desde a manipulação direta de fluxos de bytes até a abstração sofisticada de procedimentos e objetos remotos.

A adoção da Arquitetura Hexagonal com o padrão Strategy demonstrou-se fundamental para desacoplar a regra de negócio das especificidades de cada protocolo, permitindo que os mesmos contratos fossem testados sob diferentes transportes com zero impacto na camada de interface com o usuário. Finalmente, a containerização com Docker assegurou a consistência do ambiente de desenvolvimento no moderno JDK 26, garantindo a rápida validação das três abordagens de sistemas distribuídos.